

SD TCL - The Terminal and the Session

Screen size, terminal behaviour, silence, the inactivity timer, and writing to the error log.

Two different things are called the terminal. **term** is the page — how wide and how deep SD thinks your screen is, which decides where output wraps and when it pauses. **pterm** is the wire — how bytes are treated on the way in and out. They are set separately and are almost never confused once you have seen both.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** something you type as it stands.

The page: **term**

<code>term</code>	display the settings
<code>term {width} {, lines} {type}</code>	set them
<code>term colour {bgc}{,fgc}</code>	set display colour
<code>term default</code>	back to the defaults
<code>term display</code>	show the terminfo bindings

```
:term
Page width: 200
Page depth: 9999
Device      : windows
```

Depth is what decides paging. A listing longer than the page pauses; set the depth very large and it does not. That is why scripts driving SD open with something like `term 200,9999` — a script has nobody to press a key at a `Press any key` prompt, and a paged listing down a pipe stops for ever.

Width sets only the width, and the two are independent:

```
:term 132
:term
Page width: 132
Page depth: 9999
```

The depth stayed at 9999 because only the width was given.

SD'S DEFAULT PAGE IS 120 × 36

Not 80 × 24. SD's default terminal size is **120 columns by 36 lines**, and it is not a cosmetic choice: the shipped `@` dictionary records and the default `list` report layouts are formatted for that width. A report that looks wrapped or truncated on a narrow window is usually the window, not the report.

Where the number comes from, in order, when a session starts:

-
- 1 the `LINES` and `COLUMNS` environment variables, if they are numeric
 - 2 otherwise the terminfo entry's `lines` and `cols`
 - 3 **otherwise 36 and 120**
-

and whatever comes out is then raised to the minimum of 10 lines by 20 columns if it is smaller. So an ssh session or a console window normally gets its real size, and **120 × 36 is what SD falls back to when nothing else answers** — a phantom, a piped script, a service.

`term default` **puts that pair back, and it prints nothing when it does.** The verb sets the same 120 × 36 the login path falls back to, then returns without a message — so the way to see that it worked is a bare `term` after it, not the command itself:

```
:term default :term Page width: 120 Page depth: 36
```

`term 120,36` does the same thing by hand and is worth knowing for the cases where you want some other size back.

`Device` is the terminfo entry in use, `windows` on this port. If a terminal type has no terminfo entry SD falls back to `windows` rather than failing, so this line is worth reading when screen handling misbehaves.

The wire: `pterm`

```
:pterm display
Break trapping: On (char ^C)
Case inversion: Off
Input newline: CR
Output newline: CRLF
Binary mode (client to server): Off
Binary mode (server to client): Off
Telnet negotiation: Off
```

`pterm` **with no keyword prints nothing.** It is not a display verb that has gone quiet — it takes a keyword and `display` is one of them. The full set:

<code>pterm display</code>	the settings above
<code>pterm lptr</code>	send them to the default printer
<code>pterm break on</code> <code>off</code> <code>c</code>	enable, disable, or set the break character
<code>pterm case invert</code> <code>noinvert</code>	case inversion on input
<code>pterm newline cr</code> <code>lf</code> <code>crLf</code>	the output line ending
<code>pterm return cr</code> <code>lf</code>	the input line ending
<code>pterm binary on</code> <code>off</code>	pass bytes through untranslated
<code>pterm telnet on</code> <code>off</code>	recognise <code>TN_IAC</code>
<code>pterm reset</code> <i>string</i>	the terminal reset string
<code>pterm prompt</code> <code>":"</code> <code>:::"</code>	change the command prompt and the continuation prompt

Input `CR`, **output** `CRLF` **is the shipped setting** and it matches what SD writes to everything else externally readable on this port.

`pterm break off` **disables the break key for the session.** It is the right thing before a job that must not be interrupted and the wrong thing to leave set, because it is also how you stop a runaway program.

Silence: `hush`

```
hush on
hush off
hush expression
```

`hush on` **discards output entirely** — it does not redirect it, and nothing is kept. With no argument it toggles.

It suppresses the command echo as well as the output, which is more thorough than people expect. Below, `who` is typed **twice** — once inside the silence and once after it — and the transcript contains one:

```
:hush on
:who
```

```
14 DON
:term 132
```

The first `who`, and `hush off` itself, produced no line at all. **If you are debugging a script that has gone quiet, an unmatched `hush on` looks exactly like a hang.**

The bell

```
bell on
bell off
```

Sets or clears the character SD sends to sound the terminal bell. **Neither form prints anything**, and `bell` with no keyword is an error rather than a toggle: *Mode keyword missing or invalid*.

Clearing type-ahead and stored prompts

```
clearinput      clear.input
clearprompts    clear.prompts
```

`clearinput` discards anything typed ahead but not yet read. `clearprompts` discards the inline prompt and response text a previous command left behind. Neither prints anything.

Each has two spellings and they are the same verb. `clear.input` and `clear.prompts` are separate VOC records pointing at the same internal verb number as the run-together forms, which is the pattern described on SD TCL - The Command Processor. Use whichever reads better; nothing distinguishes them.

Neither can be demonstrated down a pipe, and `clearinput` should not be sent down one at all. In a piped session the input stream *is* the script, so discarding unread input discards the commands that have not run yet — including the `off` that would have ended the session. **A run that did this hung, was killed, and left a session in the user table that only an elevated `sd -cleanup` could clear.** Both verbs are for a person at a terminal.

The inactivity timer

```
autologout
```

```
autologout n
```

```
:autologout  
Autologout is disabled
```

which is the shipped state. Setting a timer ends a session that has been idle too long — useful on a machine where people walk away from a prompt, and worth thinking about before enabling on a machine where a long-running interactive job is normal.

The command echo: `echo`

```
echo on  
echo off  
echo
```

`echo off` **stops SD echoing what you type**; with no keyword it toggles. Neither form prints anything, and `echo off` suppresses its own confirmation along with everything else.

```
:echo off  
:28 DON  
:who  
28 DON
```

That is one `who` under `echo off` and one after `echo on`. **The first has no `:who` line above it** — its output ran straight on after the prompt — and `echo on` produced no line of its own either, because the echo it restores does not apply to the command that restores it.

It is not `hush`, and confusing the two wastes time. `echo` decides whether your **input** is shown back; `hush` decides whether SD's **output** is shown at all. A session that has gone completely silent has `hush on` set; one where the commands vanish but the answers still appear has `echo off`.

Dates and times

date	today, spelled out
date internal	today as SD's internal day number
date n	day number n, spelled out
date some-date	that date's day number

time	the time and the date
time internal	seconds since midnight

```
:time
14:10:34 27 AUG 2026
:date
Thursday, 27 August 2026 02:10pm
```

`date` **prints the time and** `time` **prints the date**. Both print both, in different formats and a different order, and neither name says so. `date` is the long spelled-out form and `time` is the short one.

`internal` on either gives the raw number a program would see:

```
:date internal
21424
:time internal
51088
```

the day number and seconds since midnight. `date` **with an argument is a converter, and which way it converts depends on what you give it** — a bare number is read as a day number and comes back as a date, anything else is read as a date and comes back as a day number:

```
:date 20000
Monday, 03 October 2022
:date 4 jul 2026
21370
:date zznotadate
Invalid date format
```

`time` takes no argument other than `internal`.

WHICH WAY ROUND A DATE IS READ: `DATE.FORMAT`

```
date.format on | off | display
```

```
:date.format display
European date format is off
```

`on` selects European (day-first) date format for the session, `off` returns to the default, and `display` reports which is in force. If a default conversion code other than `D` is set, `display` names that too.

`date.format` **with no keyword prints nothing** — like `pterm`, it wants one.

Setting the machine's date is a different thing entirely — it is an administrator verb, it changes the clock for the whole installation rather than for your session, and it is in the **administrator documentation** under *Accounts and Security*.

Waiting

```
sleep n
sleep hh:mm{:ss}
```

A number is seconds; a time is *until* that time, rolling over midnight if it has already passed. `sleep` **with no argument returns immediately** rather than sleeping for ever — the argument parses as zero.

Writing to the error log

```
logmsg text
```

Everything after the verb is the message. It prints nothing, and what it writes is a stamped, attributed entry in the system error log:

```
27 Aug 26 13:13:31 User 11 (pid 1786, don):
    nightly run finished
```

The user number, the process id and the account name are added for you, so a message does not need to say who wrote it. The log is `C:\ProgramData\SD\sdsys\errlog`, shared by every session, which makes `logmsg` the right way for a phantom or a scheduled job to report something a person will read later — there is no terminal to print to and no transcript kept.

See also

SD TCL - The Command Processor · SD TCL - Printing and Spooling · SD Basic - Terminal Input and Output.
